

Suppose your producer sends a message:

```
kafkaTemplate.send("orders", "customer1", "Order Created");
```

The producer successfully sends the request to Kafka, **but immediately after writing the message, the broker crashes before sending the ACK back to the producer.**

### Questions:

1. What will the producer do?
2. Will the producer retry automatically?
3. Can duplicate messages occur?
4. Which producer configurations control this behavior?

**Answers:** "When the producer sends a message, it waits for an acknowledgment based on the acks configuration. If the broker crashes before sending the ACK, the producer assumes the request may have failed and retries the send if retries are enabled. Since the broker may have already written the message before crashing, a retry can create duplicate messages. To prevent duplicates, we enable the idempotent producer (enable.idempotence=true). If the leader broker crashes, Kafka elects a new leader from the in-sync replicas, and the producer continues sending to the new leader after refreshing metadata."

**Question:** what happen if any of broker crashes?

**Answer :**

"When the leader broker fails, Kafka's controller detects the failure and elects a new leader from the In-Sync Replicas (ISR). Only replicas that are fully synchronized with the leader are eligible. After leader election, producers refresh metadata and continue sending messages to the new leader."

Suppose you have:

Topic = orders

Partitions = 3

Consumer Group = order-group

Consumers = 5

### Question:

- How many consumers will actually consume messages?
- What happens to the remaining consumers?
- Why?

**Answers :** "If a topic has 3 partitions and the consumer group has 5 consumers, only 3 consumers will be assigned partitions because one partition can be consumed by only one consumer within the same consumer group. The remaining 2 consumers stay idle. If an active consumer fails, the Group Coordinator detects the missing heartbeats and triggers a rebalance. During rebalancing, Kafka recalculates partition assignments across all consumers in the group, and one of the previously idle consumers may receive the unassigned partition."

Suppose your consumer processes a message:

Read Message



Update Database



✗ Consumer crashes



Offset NOT committed

### Questions:

1. What happens when the consumer starts again?
2. Will Kafka deliver the same message again?
3. Is this **At Most Once**, **At Least Once**, or **Exactly Once** delivery?

**Answer :** "If the consumer crashes before committing the offset, Kafka considers that message unprocessed because the committed offset was not updated. After the consumer restarts, or after another consumer receives the partition during rebalancing, consumption resumes from the last committed offset. As a result, the same message is delivered again. This behavior provides **at-least-once delivery**, because a message may be processed more than once unless the application is idempotent."

If the consumer crashes after updating the database but before committing the Kafka offset, Kafka will redeliver the message. To avoid duplicate database updates, the application should be idempotent. We typically use a unique business key such as Order ID or Payment ID, or enforce a unique constraint in the database. `enable.idempotence=true` prevents duplicate writes from the producer to Kafka, but it does not prevent duplicate processing by the consumer."

**Question :**

Suppose you have:

Topic = orders

Partitions = 3

Replication Factor = 3

The leader for **Partition 1** crashes.

**Answer :**

"ISR stands for In-Sync Replicas. These are replicas that are fully synchronized with the leader. If the leader broker for a partition crashes, the Kafka Controller detects the failure and elects one of the replicas in the ISR as the new leader. Producers refresh their metadata and continue sending messages to the new leader."

"If all ISR replicas are unavailable and `unclean.leader.election.enable=false`, Kafka cannot elect a new leader. The partition becomes unavailable, so producers cannot write and consumers cannot read from that partition. If `unclean.leader.election.enable=true`, Kafka may elect an out-of-sync replica as the new leader, restoring availability but risking data loss."

Suppose your producer has:

```
acks=all
enable.idempotence=true
retries=5
```

### **Question:**

Why do we still configure `retries=5` if `enable.idempotence=true` is enabled?

What problem does idempotence solve, and what problem do retries solve?

This is another question that's commonly asked in experienced Java/Kafka interviews.

### **Answers :**

"Retries and idempotence solve different problems. Retries improve reliability by allowing the producer to resend a message when it doesn't receive an acknowledgment due to a temporary failure, such as a broker crash or network issue. However, retries can create duplicate messages if the broker had already written the record before the ACK was lost. By enabling `enable.idempotence=true`, Kafka assigns sequence numbers to producer records and ignores duplicate retries. This allows us to retry safely without creating duplicate records."

When idempotence is enabled, Kafka assigns a unique Producer ID (PID) to the producer. Every message sent by that producer includes a monotonically increasing sequence number. The broker tracks the latest sequence number for each producer and partition. If a retry arrives with a sequence number that has already been processed, the broker recognizes it as a duplicate and discards it instead of writing it again.

### **Question :**

Suppose:

```
Producer
  ↓
Kafka
  ↓
Consumer
  ↓
Database
```

The database is **very slow**.

The consumer processes **100 messages per second**, but the producer sends **10,000 messages per second**.

### Question:

1. What problem will occur?
2. What is **Consumer Lag**?
3. How would you solve this problem in production?

### Answer :

Suppose:

Producer → 10,000 messages/sec

Consumer → 100 messages/sec

Kafka **doesn't reject the messages**.

Instead:

Producer



Kafka Topic

-----

10000 Messages Stored

-----



Consumer



Database (Slow)

Kafka stores the messages on disk.

The consumer continues reading as fast as it can.

## What is Consumer Lag?

Consumer Lag is:

**The difference between the latest offset written by the producer and the latest offset committed by the consumer.**

Example:

Producer Offset = 10000

Consumer Offset = 8000

Then

Lag = 2000

It means **2,000 messages** are waiting to be processed.

## Why is Kafka useful?

Without Kafka:

Producer  
↓  
Database (Slow)

The producer would block or requests would fail.

With Kafka:

Producer  
↓  
Kafka  
↓  
Consumer  
↓  
Database

The producer keeps working because Kafka acts as a **buffer** between the producer and the consumer.

## How do we solve high consumer lag?

In production, we have several options:

### 1. Increase the number of consumers

If the topic has enough partitions:

3 Consumers



10 Consumers

More consumers process messages in parallel.

### 2. Increase the number of partitions

More partitions allow more consumers to work simultaneously.

### 3. Optimize database operations

For example:

- Batch inserts
- Connection pooling
- Index optimization

### 4. Scale the consumer application

Run multiple instances of the consumer.

Suppose your topic has:

Topic = orders

Partitions = 3

You already have **1 million messages** in the topic.

Now your manager says:

"Increase the partitions from 3 to 6."

### Questions:

1. Can Kafka increase the number of partitions?
2. What happens to the **existing 1 million messages**?
3. Will the partition for customer1 remain the same after increasing partitions?
4. Can this affect message ordering?

### Answer :

"Kafka allows increasing the number of partitions, but existing messages remain in their original partitions and are not redistributed. Only new messages are assigned across the expanded set of partitions. Since Kafka calculates the destination using  $\text{hash}(\text{key}) \% \text{numberOfPartitions}$ , increasing the partition count changes the result of the calculation. As a result, future messages for the same key may go to a different partition, which can affect ordering guarantees for that key. Increasing partitions also enables higher parallelism because more consumers in the same consumer group can process partitions concurrently.

Suppose your code is:

```
updateBalance(accountId, 1000);  
commitOffset();
```

The consumer crashes **after** `updateBalance()` but **before** `commitOffset()`.

**What is the risk if `updateBalance()` simply subtracts ₹1000 from the account balance every time it runs?**

### Answer



"If the consumer crashes after updating the account balance but before committing the Kafka offset, Kafka assumes the message was not processed and delivers it again after restart. As a result, `updateBalance()` executes a second time, deducting another ₹1000. This leads to duplicate processing and inconsistent financial data. To prevent this, the business operation must be idempotent by using a unique transaction ID, maintaining a processed-events table, or implementing the Transactional Outbox pattern."

What should happen?

1. Should Kafka discard the message?
2. Should Kafka retry it?
3. What is a **Dead Letter Topic (DLT)**?
4. After how many retries should a message be sent to the DLT?

This is one of the **most frequently asked Kafka + Spring Boot interview questions**.

### **Answer**

"If the consumer fails because of a temporary issue such as a database outage, the message should not be discarded. The consumer retries processing according to the configured retry policy. If all retry attempts fail, the message is published to a Dead Letter Topic (DLT) so that normal message processing can continue without blocking the consumer. After the root cause is fixed, messages from the DLT can be reprocessed either by a dedicated DLT consumer or by republishing them to the original topic.

For temporary failures such as a database outage, I would not immediately send the message to the DLT after three quick retries. Instead, I would use retry topics with exponential backoff or delayed retries. This gives the downstream system time to recover. The DLT should be used only when retries are exhausted or when the message is permanently invalid.